

POLITECHNIKA WARSZAWSKA
WYDZIAŁ ELEKTRONIKI I TECHNIK INFORMACYJNYCH

Projekt część I

Ścieżka nowoczesna, Temat N02:

System skanowania lokalnej sieci komputerowej z wykrywaniem
potencjalnych zagrożeń oraz raportowaniem e-mail dla urządzeń sieciowych
typu router

Bezpieczeństwo systemów i oprogramowania (BSO)

Michał Sokół (337056)
Marek Zawadzki (337062)

Warszawa, 4 kwietnia 2026

Spis treści

1	Wstęp	2
2	Cel, zakres i założenia projektowe	2
2.1	Cel główny	2
2.2	Cele szczegółowe	2
2.3	Wymagania funkcjonalne	3
2.4	Wymagania niefunkcjonalne	3
2.5	Zakres funkcjonalny	4
2.6	Ograniczenia środowiskowe	4
2.7	Ograniczenia i decyzje bezpieczeństwa	4
3	Architektura rozwiązania	5
3.1	Architektura systemu	5
3.2	Role komponentów	6
3.3	Przepływ danych i interfejsy	7
3.4	Kanały komunikacji i artefakty danych	8
4	Opis komponentów systemu	9
4.1	Moduł konfiguracji i profili	9
4.2	Moduł bootstrapu i wdrożenia	11
4.3	Moduł discovery	12
4.4	Moduł orkiestracji Nmap/NSE	12
4.5	Moduł parsera i normalizacji wyników	13
4.6	Moduł klasyfikacji ryzyka	13
4.7	Moduł raportowania e-mail	14
5	Algorytmy działania	15
5.1	Algorytm główny systemu	15
5.2	Pseudokod algorytmu głównego	16
5.3	Pseudokod klasyfikacji ryzyka	17
5.4	Pseudokod bootstrapu wdrożeniowego	18
6	Propozycja implementacji	18
6.1	Dobór technologii	18
6.2	Proponowana struktura aplikacji	19
6.3	Szkic implementacyjny w Go	20
6.4	Propozycja lekkiego obrazu kontenera	22
7	Scenariusz działania systemu	23
8	Ocena wykonalności rozwiązania	23
9	Podsumowanie i zadeklarowane cele do osiągnięcia	24

1 Wstęp

Współczesna sieć domowa coraz rzadziej ogranicza się do komputera i routera. Typowe środowisko użytkownika końcowego obejmuje obecnie telefony, komputery przenośne, telewizory Smart TV, kamery IP, drukarki, konsole, urządzenia IoT i automatyki domowej. Taka różnorodność zwiększa powierzchnię ataku oraz utrudnia bieżące monitorowanie stanu bezpieczeństwa, zwłaszcza gdy użytkownik nie posiada kompetencji administracyjnych ani narzędzi do regularnego audytu (Mosenia i Jha 2017; Fernandes, Jung i Prakash 2016; Everson i Cheng 2024; Vardakis i inni 2024; Kumar i inni 2019).

Z perspektywy operatora telekomunikacyjnego naturalnym kierunkiem rozwoju usług jest przeniesienie części funkcji diagnostycznych i ochronnych na urządzenie brzegowe klienta, czyli router. Standardy operatorskiego zarządzania urządzeniami CPE, takie jak TR-069 oraz nowszy USP/TR-369, przewidują zdalne zarządzanie, diagnostykę i wdrażanie usług pomocniczych na urządzeniach końcowych (Broadband Forum marzec 2018, czerwiec 2025). Z kolei RouterOS 7 na wybranych architekturach wspiera uruchamianie kontenerów, co otwiera możliwość instalacji lekkich usług bezpieczeństwa bez konieczności używania dodatkowego serwera (MikroTik 2026b). Rola operatora jako podmiotu pośredniczącego w diagnozie i rekomendowaniu działań naprawczych w środowisku smart home znajduje również potwierdzenie w badaniach nad rzeczywistymi interwencjami bezpieczeństwa w ekosystemach urządzeń konsumenckich (Bouwmeester i inni 2021).

Celem niniejszego projektu jest zaprojektowanie lekkiego systemu skanowania lokalnej sieci komputerowej uruchamianego w kontenerze na routerze MikroTik. System ma korzystać z narzędzia Nmap oraz skryptów NSE, wykrywać aktywne hosty i potencjalne zagrożenia, a następnie generować raport e-mail dla administratora lub operatora (Nmap Project 2026a, 2026b). Projekt został przygotowany zgodnie z podejściem *od ogółu do szczegółu*: od architektury systemu i komunikacji między komponentami, przez szczegółowy opis modułów, aż po algorytmy, pseudokod i propozycję implementacji.

2 Cel, zakres i założenia projektowe

2.1 Cel główny

Celem głównym projektu jest opracowanie architektury rozwiązania umożliwiającego automatyczne, niskointerwencyjne i możliwie transparentne skanowanie lokalnej sieci użytkownika z poziomu routera operatorskiego, bez konieczności instalowania osobnego komputera lub ręcznego uruchamiania narzędzi diagnostycznych.

2.2 Cele szczegółowe

- zaprojektowanie lekkiego agenta skanującego uruchamianego w kontenerze,
- wykorzystanie Nmap do wykrywania hostów, usług i podstawowych anomalii konfiguracyjnych,

- wykorzystanie profili NSE dobieranych zależnie od scenariusza operacyjnego do pogłębionej analizy usług,
- automatyzacja harmonogramu skanów i raportowania e-mail,
- ograniczenie wymagań zasobowych tak, aby rozwiązanie było możliwe do uruchomienia na routerze MikroTik,
- przygotowanie rozwiązania do publikacji w repozytorium GitHub oraz do zdalnej instalacji za pomocą jednej komendy wykonywanej przez SSH.

2.3 Wymagania funkcjonalne

System powinien realizować następujące funkcje:

- automatyczne wykrywanie aktywnych hostów w zadanej podsieci LAN,
- skanowanie portów, usług i podstawowych cech ekspozycji sieciowej na wykrytych hostach,
- możliwość rozszerzenia skanu o zestawy skryptów NSE zgodne z wybranym profilem,
- klasyfikację wyników skanowania pod kątem potencjalnych zagrożeń,
- zapis surowych i przetworzonych wyników w formatach odpowiednich do dalszej analizy, w szczególności XML, JSON oraz raportu HTML,
- wysyłkę raportu e-mail do zdefiniowanego odbiorcy z wykorzystaniem zewnętrznego serwera SMTP,
- pracę konfigurowaną z pliku `config.yaml` lub równoważnego zestawu zmiennych środowiskowych.

2.4 Wymagania niefunkcjonalne

Poza zakresem funkcjonalnym projekt powinien spełniać również wymagania jakościowe i wdrożeniowe:

- możliwie mała objętość rozwiązania i niewielkie zużycie pamięci,
- możliwość uruchomienia w lekkim kontenerze na routerze MikroTik,
- odporność na błędy sieciowe, niepełne wyniki skanowania i błędy zewnętrznych narzędzi,
- czytelność raportu dla odbiorcy niezaawansowanego technicznie,
- łatwość rozbudowy o nowe reguły ryzyka, profile skanowania oraz dodatkowe kanały raportowania.

2.5 Zakres funkcjonalny

Projektowane rozwiązanie obejmuje:

1. pobranie konfiguracji i uruchomienie agenta w kontenerze,
2. cykliczne skanowanie zadanej podsieci lokalnej,
3. identyfikację aktywnych hostów, otwartych portów i wersji usług,
4. wykonanie testów z użyciem skryptów NSE zgodnych z aktywnym profilem skanowania,
5. agregację i normalizację wyników,
6. klasyfikację ryzyka według prostego modelu punktowego,
7. wygenerowanie raportu tekstowego lub HTML,
8. wysłanie raportu e-mail do operatora lub administratora.

2.6 Ograniczenia środowiskowe

Uruchomienie systemu bezpośrednio na routerze narzuca istotne ograniczenia dotyczące pamięci operacyjnej, przestrzeni dyskowej oraz czasu wykonywania zadań. Zasoby sprzętowe urządzeń MikroTik używanych w środowiskach domowych i małych biur są wyraźnie zróżnicowane: przykładowo model hAP ac² oferuje 128 MB pamięci RAM i 16 MB pamięci FLASH, nowszy hEX refresh 512 MB RAM i 128 MB NAND, natomiast hAP ax² 1 GB RAM i 128 MB NAND (MikroTik 2026f, 2026h, 2026g). Oznacza to, że nawet jeśli wybrane modele dysponują relatywnie dużą pamięcią operacyjną, trwała przestrzeń na obraz kontenera, profile skanowania, raporty i pliki pośrednie nadal może stanowić realne ograniczenie wdrożeniowe.

Z tego względu projekt zakłada użycie lekkiego obrazu kontenera, ograniczenie liczby zależności oraz delegowanie właściwej logiki bezpieczeństwa do pojedynczej aplikacji uruchamianej wewnątrz kontenera. W praktyce oznacza to konieczność ostrożnego doboru profili skanowania, czasu wykonywania zadań oraz miejsca przechowywania danych tymczasowych. W zależności od konkretnego modelu routera i dostępnej pamięci masowej możliwe jest również wykorzystanie zewnętrznego nośnika danych lub innego rozszerzenia przestrzeni roboczej, jeśli wymaga tego środowisko testowe (MikroTik 2026b, 2026c).

2.7 Ograniczenia i decyzje bezpieczeństwa

Projekt nie zakłada całkowitej rezygnacji z agresywnych testów penetracyjnych, ponieważ jednym z jego celów jest możliwość przeprowadzenia dokładnej analizy bezpieczeństwa urządzeń obecnych w sieci lokalnej. Dokumentacja Nmap wskazuje jednak, że kategorie skryptów NSE obejmują również tryby *intrusive*, *exploit*, *dos* oraz *brute*, które mogą wpływać na stabilność badanych usług lub urządzeń (Nmap Project 2026b, 2026c). Z tego względu w projekcie przyjęto model profilowy, w którym poziom intensywności testów jest dobierany zależnie od celu analizy.

Przyjęto następujące założenia bezpieczeństwa i organizacji testów:

- skanowanie ogranicza się wyłącznie do sieci lokalnych przypisanych klientowi lub do innej sieci, dla której uzyskano formalną zgodę na testy,
- system obsługuje co najmniej trzy klasy profili: diagnostyczny, pogłębiony oraz pentestowy,
- profil diagnostyczny może być wykonywany cyklicznie w sposób zautomatyzowany,
- profile obejmujące skrypty `intrusive`, `exploit` lub `dos` powinny być uruchamiane wyłącznie świadomie przez administratora lub operatora, najlepiej w kontrolowanym oknie serwisowym,
- system nie wprowadza automatycznych zmian konfiguracyjnych na urządzeniach końcowych,
- wyniki mają charakter diagnostyczny i wspomagający, natomiast decyzje eksploatacyjne i naprawcze pozostają po stronie administratora.

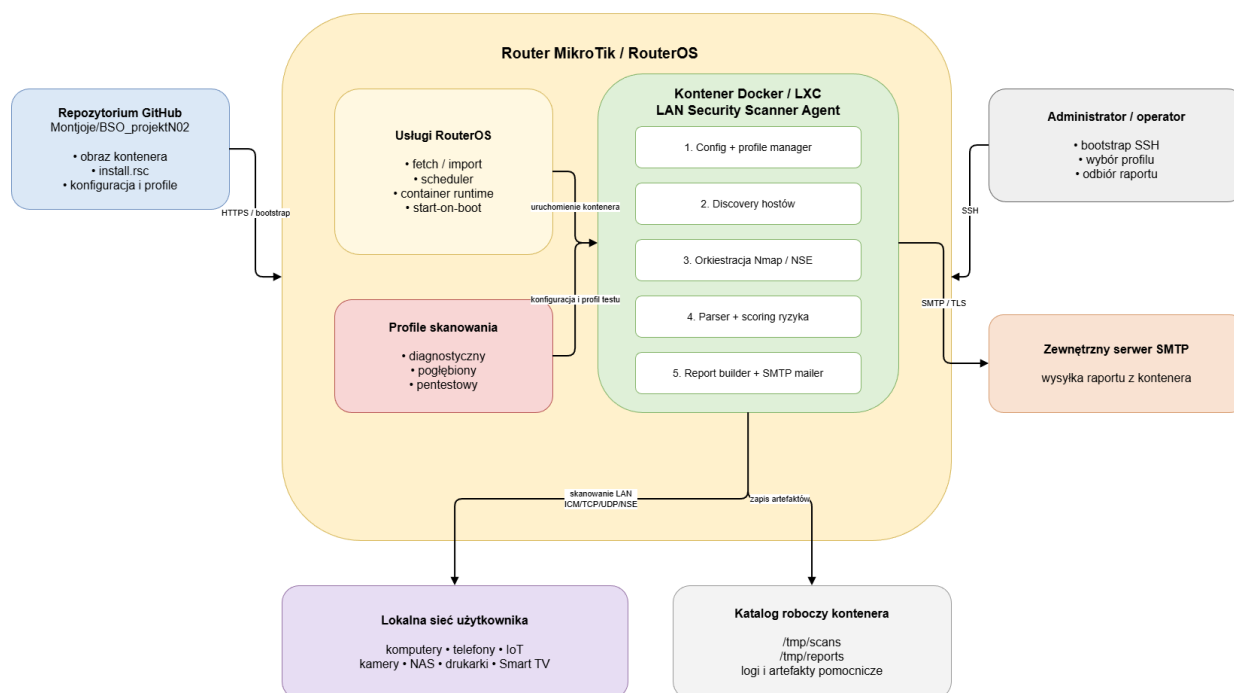
Istotnym ograniczeniem wdrożeniowym platformy MikroTik jest to, że obsługa kontenerów jest domyślnie wyłączona i wymaga fizycznego dostępu do urządzenia przy pierwszym włączeniu tej funkcji (MikroTik 2026b, 2026c). Oznacza to, że w praktyce model wdrożenia obejmuje jednorazowe przygotowanie routera przez operatora, a następnie dalsze zarządzanie już zdalnie.

3 Architektura rozwiązania

3.1 Architektura systemu

Rysunek 1 przedstawia architekturę projektowanego systemu w ujęciu łączącym perspektywę logiczną i wdrożeniową..

Architektura wdrożeniowa i uruchomieniowa systemu



Rysunek 1: Architektura systemu w ujęciu logiczno-wdrożeniowym.

System składa się z warstwy zarządzania i wdrożenia, warstwy infrastrukturalnej oraz warstwy funkcjonalnej. Repozytorium GitHub pełni rolę źródła obrazu kontenera, pliku `install.rsc`, konfiguracji oraz profili skanowania. Administrator lub operator inicjuje wdrożenie i może nadpisywać profil pracy systemu, komunikując się z routerem MikroTik z wykorzystaniem SSH.

Router MikroTik z systemem RouterOS pełni rolę hosta uruchomieniowego. Odpowiada za pobranie artefaktów z repozytorium, import konfiguracji, uruchomienie kontenera, ustawienie harmonogramu zadań oraz zarządzanie startem rozwiązania po restarcie urządzenia (MikroTik 2020, 2026e, 2026b). Wewnątrz kontenera działa agent *LAN Security Scanner Agent*, obejmujący moduły konfiguracji i profili, discovery hostów, orkiestracji Nmap/NSE, parsera i oceny ryzyka oraz budowy raportu i wysyłki wiadomości e-mail.

Przyjęta architektura zakłada także istnienie lokalnej sieci użytkownika jako środowiska obserwowanego oraz zewnętrznego serwera SMTP jako kanału wyjściowego dla raportów. W projekcie docelowym raport wysyłany jest bezpośrednio z kontenera, natomiast mechanizm `/tool e-mail` RouterOS pozostaje wariantem alternatywnym (MikroTik 2026d).

3.2 Role komponentów

Każdy z głównych komponentów systemu pełni odrębną funkcję:

- **Repozytorium GitHub** przechowuje obraz kontenera, skrypt bootstrapujący, konfigurację oraz profile skanowania i stanowi punkt dystrybucji rozwiązania.

- **Administrator / operator** inicjuje wdrożenie, wybiera profil testu oraz odbiera raport końcowy.
- **Usługi RouterOS** odpowiadają za pobranie artefaktów, harmonogramowanie zadań i uruchomienie kontenera.
- **Kontener skanera** realizuje właściwą logikę bezpieczeństwa: wykrywanie hostów, skanowanie, analizę wyników i raportowanie.
- **Lokalna sieć użytkownika** stanowi obszar obserwacji i obejmuje komputery, urządzenia mobilne, IoT, kamery, drukarki, serwery NAS oraz odbiorniki Smart TV.
- **Zewnętrzny serwer SMTP** realizuje wysyłkę raportu poza routerem i poza samym środowiskiem RouterOS.

W warstwie funkcjonalnej system pracuje profilowo. Profil diagnostyczny przeznaczony jest do cyklicznego monitoringu, profil pogłębiony do rozszerzonej analizy usług, natomiast profil pentestowy może uruchamiać bardziej intensywne zestawy skryptów NSE w środowisku autoryzowanym (Nmap Project 2026b, 2026c). Skala i różnorodność urządzeń IoT obserwowanych w rzeczywistych gospodarstwach domowych wskazują, że rozwiązania przeznaczone do monitoringu sieci lokalnej powinny zakładać heterogeniczność urządzeń, usług i producentów (Kumar i inni 2019).

3.3 Przepływ danych i interfejsy

Przepływ działania systemu można opisać następująco:

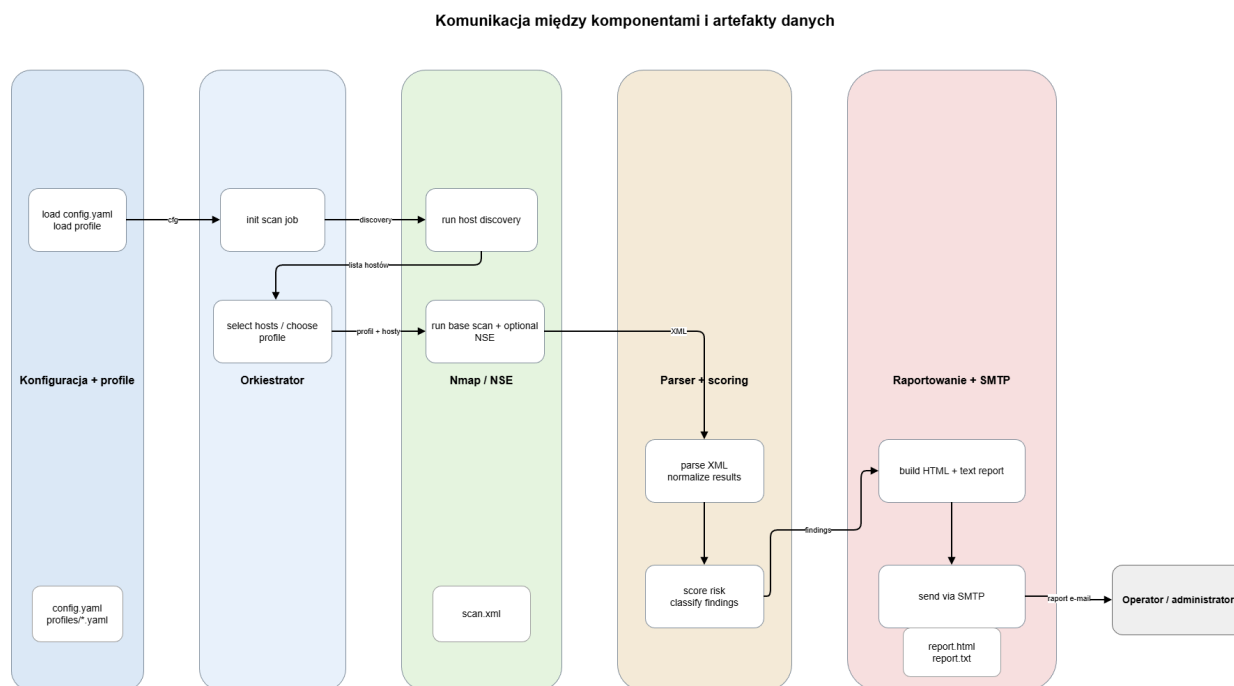
1. administrator lub operator uruchamia pojedynczą komendę bootstrap przez SSH,
2. router pobiera skrypt instalacyjny, konfigurację i obraz kontenera z repozytorium,
3. RouterOS importuje konfigurację, uruchamia kontener oraz ustawia harmonogram wykonywania zadań (MikroTik 2020, 2026e),
4. administrator lub operator wybiera profil testu: diagnostyczny, pogłębiony lub pentestowy,
5. agent pobiera konfigurację lokalną i ustala listę podsieci do skanowania,
6. agent wykonuje skan rozpoznawczy, a następnie skan właściwy zgodny z aktywnym profilem,
7. wyniki są parsowane, normalizowane i klasyfikowane,
8. raport końcowy jest wysyłany e-mailem bezpośrednio z kontenera przez zewnętrzny serwer SMTP,
9. administrator analizuje wyniki i podejmuje decyzje o dalszych działaniach.

Tabela 1: Główne interfejsy i kanały komunikacyjne.

Komponent źródłowy	Komponent docelowy	Interfejs / protokół	Cel komunikacji
Repozytorium GitHub	Router MikroTik	HTTPS / SSH	pobranie konfiguracji, skryptów i obrazu kontenera
Administrator / operator	Router MikroTik	SSH / RouterOS CLI	bootstrap wdrożenia, wybór profilu testu, uruchomienie skanu na żądanie
RouterOS scheduler	Kontener	lokalne wywołanie zadania	uruchomienie cyklicznego skanu
Kontener	Hosty LAN	ICMP, TCP, UDP	wykrywanie hostów, portów i usług
Moduł orkiestracji	Lokalny proces Nmap/NSE	CLI / proces systemowy	wykonanie skanów zgodnych z wybranym profilem
Kontener	Zewnętrzny serwer SMTP	SMTP / TLS	wysłanie raportu e-mail

3.4 Kanały komunikacji i artefakty danych

W systemie występują trzy główne rodzaje komunikacji. Pierwszy z nich to komunikacja wewnątrz aplikacji, gdzie moduły przekazują sobie dane w postaci struktur pamięciowych. Drugi to komunikacja przez artefakty pośrednie, czyli pliki konfiguracyjne, surowe wyniki skanowania oraz raporty końcowe. Trzeci rodzaj stanowi komunikacja sieciowa z hostami LAN oraz zewnętrznym serwerem SMTP.



Rysunek 2: Komunikacja pomiędzy modułami oraz główne artefakty danych.

W warstwie artefaktów danych można wyróżnić w szczególności:

- plik `config.yaml` oraz zestawy profili przechowywane w katalogu `profiles/`,
- surowe wyniki skanów zapisywane w formacie XML jako artefakty pomocnicze,
- raport końcowy w postaci HTML oraz tekstu alternatywnego przeznaczonego do wysyłki e-mail,
- opcjonalne dane stanu w katalogu `data/state/`, które mogą w przyszłości służyć do wykrywania zmian pomiędzy kolejnymi uruchomieniami.

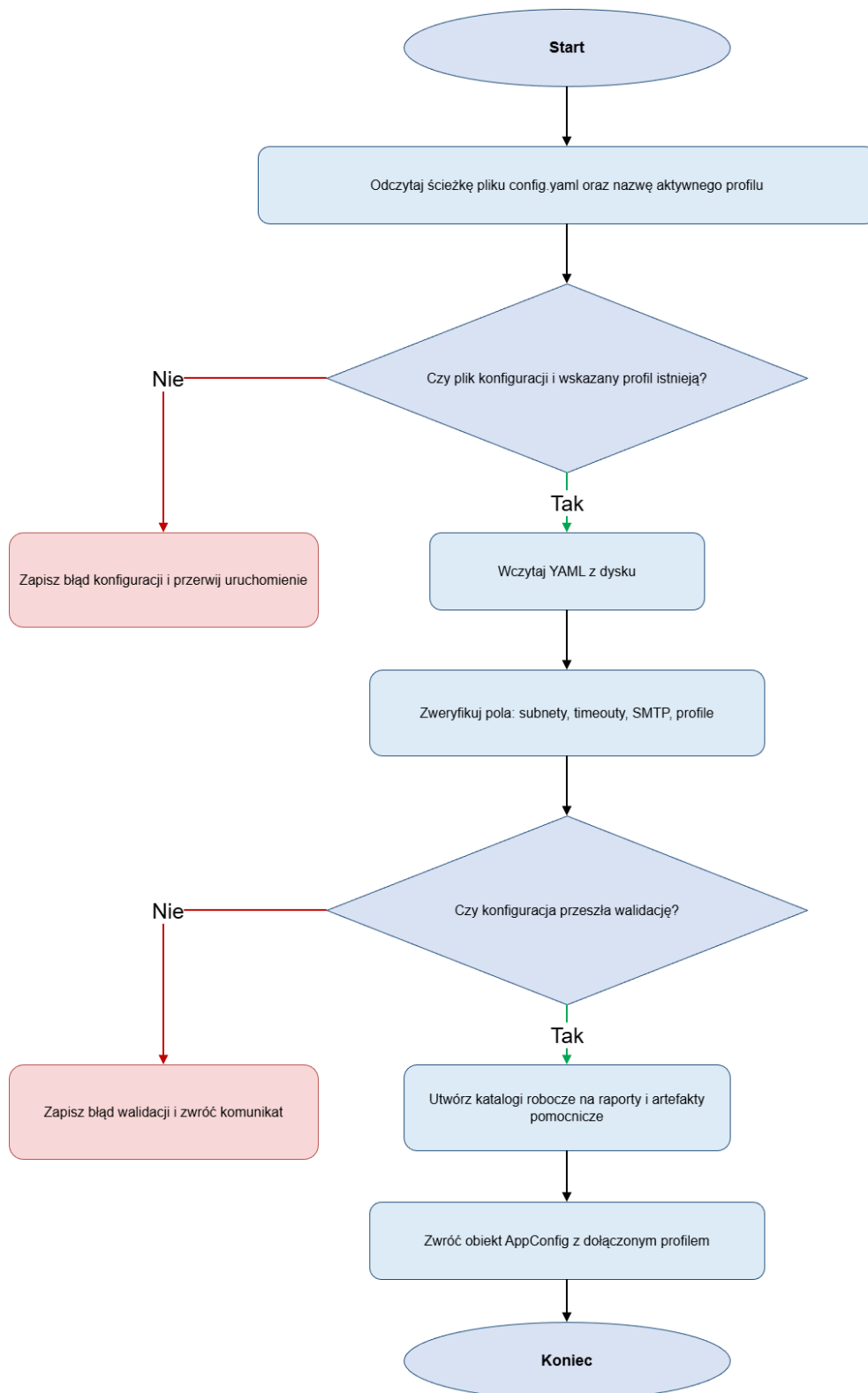
Takie rozdzielenie warstw komunikacji upraszcza debugowanie systemu, ułatwia testowanie poszczególnych modułów oraz dobrze wpisuje się w kontenerowy model wdrożenia.

4 Opis komponentów systemu

4.1 Moduł konfiguracji i profili

Moduł konfiguracji i profili odpowiada za wczytanie parametrów systemu z pliku `config.yaml`, wybór aktywnego profilu skanowania oraz walidację podstawowych ustawień środowiskowych. Jest to pierwszy komponent uruchamiany przez aplikację i jednocześnie miejsce, w którym następuje powiązanie konfiguracji operacyjnej z profilem testów: diagnostycznym, pogłębionym lub pentestowym.

Schemat blokowy modułu konfiguracji i profili



Rysunek 3: Schemat blokowy modułu konfiguracji i profili.

Z punktu widzenia implementacyjnego moduł ten powinien:

- sprawdzić obecność pliku `config.yaml` oraz wskazanego profilu,

- wczytać konfigurację z formatu YAML,
- zweryfikować poprawność kluczowych pól, takich jak podsieci, limity czasowe i parametry SMTP,
- utworzyć katalogi robocze na raporty i artefakty pomocnicze,
- zwrócić ujednolicony obiekt konfiguracyjny wykorzystywany przez pozostałe moduły.

Listing 1: Pseudokod modułu konfiguracji i profili.

```
procedure LOAD_CONFIGURATION(config_path, profile_name):
    ensure_file_exists(config_path)
    ensure_profile_exists(profile_name)

    raw_cfg <- read_yaml(config_path)
    cfg <- build_app_config(raw_cfg, profile_name)

    validate_network(cfg.network)
    validate_scanner(cfg.scanner)
    validate_mail(cfg.mail)

    ensure_output_directories(cfg.output)
    return cfg
end procedure
```

4.2 Moduł bootstrapu i wdrożenia

Jest to najprostszy komponent z perspektywy użytkownika końcowego i najważniejszy z perspektywy wymogu „jednej komendy”. Jego zadaniem jest:

- pobranie skryptu instalacyjnego z repozytorium,
- sprawdzenie, czy router ma aktywną obsługę kontenerów,
- utworzenie katalogu roboczego i miejsca na dane tymczasowe,
- pobranie obrazu kontenera,
- zdefiniowanie parametrów startowych i zasad automatycznego uruchomienia,
- zapisanie zadania do harmonogramu RouterOS.

Przykładowa komenda uruchomieniowa po stronie administratora może mieć postać:

Listing 2: Przykładowy sposób zdalnego uruchomienia bootstrapu.

```
ssh admin@ROUTER_IP \
"/tool fetch url=https://raw.githubusercontent.com/Montjoje/
BSO_projektN02/main/routeros/install.rsc \
```

```
dst-path=install.rsc; /import file-name=install.rsc"
```

Repozytorium projektu przeznaczone do publikacji rozwiązania i zdalnego wdrożenia znajduje się pod adresem https://github.com/Montjoje/BSO_projektN02. W praktycznej implementacji plik `install.rsc` powinien zostać umieszczony w katalogu `routeros/`, tak aby możliwe było jego pobranie bezpośrednio z serwisu GitHub przy użyciu jednej komendy bootstrapującej.

4.3 Moduł discovery

Moduł discovery odpowiada za wstępne rozpoznanie środowiska. W pierwszej fazie wykonuje lekki skan identyfikujący hosty aktywne w zadanej podsieci. Na tym etapie ważniejsze jest ograniczenie obciążenia niż pełna głębokość rozpoznania. Wynikiem pracy modułu jest lista aktywnych adresów IP wraz z podstawowymi metadanymi, np. czasem odpowiedzi, nazwą hosta i sygnaturą urządzenia (Nmap Project 2026a).

4.4 Moduł orkiestracji Nmap/NSE

Moduł ten jest centralnym elementem całego systemu. Odpowiada za:

- uruchamianie bazowego skanu Nmap,
- decyzję, czy dany host wymaga skanu rozszerzonego,
- dobór profilu testów zgodnie z polityką operatora,
- kontrolę limitów czasu i zasobów,
- zapisywanie wyników w ustalonym formacie XML lub zwykłego tekstu.

Nmap jest narzędziem przeznaczonym do eksploracji sieci i audytu bezpieczeństwa, a NSE umożliwia automatyzację dodatkowych testów dla wykrytych usług (Nmap Project 2026a, 2026b). W projekcie przyjęto podejście profilowe. Agent nie uruchamia dowolnych skryptów w sposób całkowicie swobodny, lecz korzysta z zestawów przypisanych do określonego trybu działania:

- **profil diagnostyczny** – skrypty informacyjne i niskoinwazyjne, używane w monitoringu cyklicznym,
- **profil pogłębiony** – rozszerzone testy usług, certyfikatów, nagłówków, protokołów i konfiguracji,
- **profil pentestowy** – zestawy testów obejmujące również kategorie `intrusive`, `exploit` i `dos`, uruchamiane wyłącznie świadomie i w środowisku autoryzowanym (Nmap Project 2026c).

W praktyce oznacza to, że ten sam agent może działać zarówno jako lekki skaner diagno-

styczny, jak i jako narzędzie dokładniejszej analizy penetracyjnej, zależnie od przyjętego scenariusza operacyjnego.

4.5 Moduł parsera i normalizacji wyników

Wyniki Nmap są bogate, ale niejednorodne. Moduł parsera odpowiada za zamianę wyjścia XML na prostszy, ujednolicony model danych. Po etapie parsowania dane mogą być wykorzystywane przez silnik oceny ryzyka, generator raportu oraz ewentualne przyszłe moduły porównujące kolejne uruchomienia.

Przykładowa struktura logiczna wpisu dla pojedynczego hosta może wyglądać następująco:

Listing 3: Przykładowy rekord po normalizacji danych.

```
{
  "ip": "192.168.88.15",
  "hostname": "kamera-garaz",
  "state": "up",
  "services": [
    {"port": 80, "proto": "tcp", "service": "http", "product": "GoAhead"},
    {"port": 554, "proto": "tcp", "service": "rtsp", "product": "unknown"}
  ],
  "findings": [
    "panel HTTP bez TLS",
    "publicznie opisany serwer RTSP"
  ]
}
```

4.6 Moduł klasyfikacji ryzyka

Moduł klasyfikacji nadaje wykrytym anomaliom wagę punktową i buduje końcową ocenę ryzyka. Rozwiązanie nie korzysta z pełnego systemu CVSS, gdyż jego celem jest szybka klasyfikacja operacyjna w środowisku domowym, a nie formalna ocena podatności dla systemów korporacyjnych. Tego rodzaju uproszczone, zautomatyzowane podejście do oceny zagrożeń dla środowisk IoT i smart home jest spójne z kierunkiem opisywanym w literaturze dotyczącej automatycznego skanowania i oceny bezpieczeństwa (Amro 2020; Duan i inni 2021; Akhilesh i inni 2022).

Przykładowe reguły mogą być następujące:

- panel administracyjny HTTP bez TLS: +20 pkt,
- wykryta stara wersja usługi lub protokołu: +20 pkt,
- obecność wielu nieuzasadnionych otwartych portów: +10 pkt,

- usługa zdalnego dostępu wystawiona wewnętrznie bez uwierzytelniania warstwowego: +15 pkt,
- urządzenie IoT z usługą znaną z częstych błędów konfiguracyjnych: +10 pkt.

Na tej podstawie system przyjmuje klasy:

- 0–19 pkt – ryzyko niskie,
- 20–39 pkt – ryzyko średnie,
- 40 pkt i więcej – ryzyko wysokie.

4.7 Moduł raportowania e-mail

Ostatni etap procesu polega na wygenerowaniu czytelnego raportu. W podstawowym wariancie raport zawiera:

- datę i czas skanu,
- przeskanowane podsieci,
- liczbę wykrytych hostów,
- listę urządzeń o najwyższym poziomie ryzyka,
- listę najważniejszych ustaleń,
- krótkie rekomendacje naprawcze.

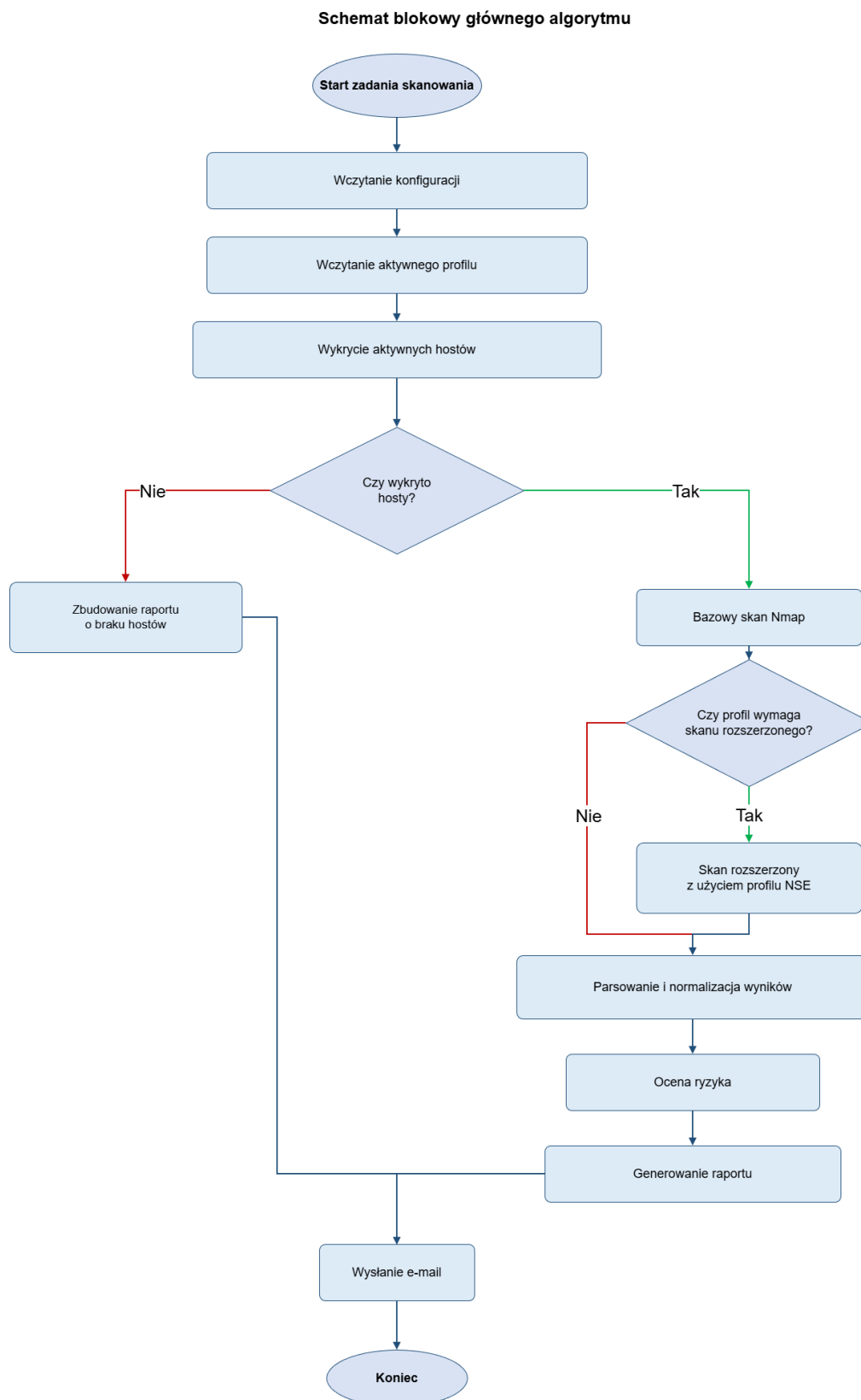
W RouterOS dostępne jest wbudowane narzędzie `/tool e-mail`, pozwalające wysyłać wiadomości e-mail z routera z wykorzystaniem uwierzytelniania prostego oraz TLS (MikroTik 2026d). W projekcie docelowo przyjęto wariant wysyłki realizowanej bezpośrednio z kontenera przez zewnętrzny serwer SMTP, ponieważ upraszcza on testowanie rozwiązania, zmniejsza zależność od konfiguracji RouterOS i pozwala utrzymać całą logikę raportowania po stronie aplikacji. Mechanizm pocztowy RouterOS pozostaje wariantem rezerwowym, który można wykorzystać w późniejszym etapie wdrożenia.

Poza samą wysyłką wiadomości system powinien zachowywać także lokalne artefakty raportowe. Surowe wyniki skanowania mogą być przechowywane w formacie XML, znormalizowane dane w formacie JSON, a raport końcowy w formie HTML i wersji tekstowej. Taki podział ułatwia zarówno ręczną analizę wyników, jak i ewentualną dalszą rozbudowę systemu o porównywanie kolejnych uruchomień lub eksport do innych kanałów raportowania.

5 Algorytmy działania

5.1 Algorytm główny systemu

Rysunek 4 przedstawia główny algorytm działania agenta. W aktualnym modelu system pracuje w trybie profilowym, co oznacza, że przebieg skanowania zależy od aktywnego profilu testu. W każdym przypadku wykonywane jest wczytanie konfiguracji, wczytanie aktywnego profilu, identyfikacja hostów oraz skan bazowy, natomiast dalszy zakres analizy zależy od tego, czy uruchomiono profil diagnostyczny, pogłębiony czy pentestowy.



Rysunek 4: Schemat blokowy głównego algorytmu działania systemu.

5.2 Pseudokod algorytmu głównego

Listing 4: Pseudokod głównego zadania skanowania.

```
procedure RUN_SCAN_JOB():
    config <- load_config()
    profile <- load_active_profile(config)
    targets <- discover_hosts(config.subnets)

    if targets is empty:
        report <- build_empty_report(config)
        send_report(report)
        store_log(report)
        return

    base_results <- run_base_scan(targets, profile)
    parsed_base <- parse_results(base_results)

    if profile.mode in {DEEP, PENTEST}:
        extended_results <- run_extended_scan(parsed_base.hosts,
            profile)
        normalized <- normalize(base_results, extended_results)
    else:
        normalized <- normalize(base_results, [])

    findings <- classify_findings(normalized, config.rules)
    risk <- score_risk(findings)
    report <- build_report(findings, risk, profile)

    send_report(report)
    store_log(report)
end procedure
```

5.3 Pseudokod klasyfikacji ryzyka

Listing 5: Pseudokod uproszczonego silnika punktowego.

```
procedure SCORE_RISK(host):
    points <- 0

    if host.has_http_admin and not host.uses_tls:
        points <- points + 20
    if host.has_legacy_protocol:
        points <- points + 20
    if host.open_ports > threshold:
        points <- points + 10
    if host.exposes_remote_access:
        points <- points + 15
```

```

    if host.is_iot and host.has_common_misconfig:
        points <- points + 10

    if points >= 40:
        return HIGH
    else if points >= 20:
        return MEDIUM
    else:
        return LOW
end procedure

```

5.4 Pseudokod bootstrapu wdrożeniowego

Listing 6: Pseudokod jednorazowego przygotowania urządzenia.

```

procedure INSTALL_AGENT():
    verify_routeros_version()
    verify_container_support()
    create_workdir()
    download_container_image()
    write_env_config()
    create_container_definition()
    enable_start_on_boot()
    create_scheduler_entry()
    log_success()
end procedure

```

6 Propozycja implementacji

6.1 Dobór technologii

Wariant rekomendowany dla implementacji prototypu:

- **język sterujący:** Go 1.26.1 – wysoka wydajność, mały narzut pamięciowy, kompilacja do pojedynczego pliku binarnego oraz dobra obsługa XML/JSON i SMTP,
- **silnik skanowania:** Nmap – uruchamiany jako proces zewnętrzny z parametrami kontrolowanymi przez aplikację,
- **system kontenera:** obraz oparty o Alpine Linux – mały rozmiar i prosty proces budowania,
- **konfiguracja:** plik YAML lub zmienne środowiskowe,
- **raportowanie:** HTML + tekst alternatywny, wysyłane przez SMTP bezpo-

średnio z kontenera.

Dobór języka Go wynika z tego, że w części projektowej istotne są nie tylko czytelność kodu i łatwość integracji z narzędziami systemowymi, ale również możliwość przygotowania bardzo lekkiego i wydajnego komponentu uruchamianego wewnątrz kontenera. Go dobrze nadaje się do budowy aplikacji systemowych, które sterują zewnętrznymi procesami, przetwarzają dane XML oraz generują raporty bez konieczności instalowania dodatkowego interpretera. Dzięki temu rozwiązanie lepiej wpisuje się w wymagania projektu związane z ograniczonymi zasobami urządzenia brzegowego.

6.2 Proponowana struktura aplikacji

Listing 7: Sugerowana struktura repozytorium.

```
lan-security-scanner/  
|-- container/  
|   |-- Dockerfile  
|   '-- entrypoint.sh  
|-- cmd/  
|   '-- scanner/  
|       '-- main.go  
|-- internal/  
|   |-- config/  
|   |   '-- config.go  
|   |-- discovery/  
|   |   '-- discovery.go  
|   |-- scanner/  
|   |   '-- scanner.go  
|   |-- parser/  
|   |   '-- parser.go  
|   |-- scoring/  
|   |   '-- scoring.go  
|   |-- reporting/  
|   |   '-- reporting.go  
|   |-- mailer/  
|   |   '-- mailer.go  
|   '-- models/  
|       '-- models.go  
|-- configs/  
|   '-- config.example.yaml  
|-- profiles/  
|   |-- baseline.yaml  
|   |-- deep.yaml  
|   '-- pentest.yaml  
|-- templates/  
|   '-- report.html
```

```
|-- routeros/
|   '-- install.rsc
|-- README.md
|-- go.mod
|-- go.sum
|-- BS026L_PRO_etapI_MS_MZ.pdf
'-- BS026L_PRO_etapI_MS_MZ.zip
```

Przedstawiona struktura repozytorium została uzupełniona o plik przykładowej konfiguracji. W czasie działania aplikacja może dodatkowo tworzyć katalogi robocze, takie jak `data/scans/`, `data/reports/` oraz `data/state/`, jeśli zostaną wskazane w konfiguracji.

6.3 Szkic implementacyjny w Go

Listing 8: Przykładowy szkic struktury orkiestratora.

```
package scanner

import (
    "fmt"
    "os/exec"
    "path/filepath"
)

type ScanProfile struct {
    Name          string
    NmapArgs      []string
    NSEScripts    []string
}

type ScanOrchestrator struct {
    OutputDir string
}

func NewScanOrchestrator(outputDir string) *ScanOrchestrator {
    return &ScanOrchestrator{OutputDir: outputDir}
}

func (s *ScanOrchestrator) RunProfile(target string, profile
    ScanProfile) (string, error) {
    outFile := filepath.Join(s.OutputDir, fmt.Sprintf("%s-%s.xml",
        target, profile.Name))

    args := append([]string{}, profile.NmapArgs...)
    if len(profile.NSEScripts) > 0 {
```

```

        args = append(args, "--script", joinScripts(profile.
            NSEScripts))
    }
    args = append(args, "-oX", outFile, target)

    cmd := exec.Command("nmap", args...)
    if err := cmd.Run(); err != nil {
        return "", err
    }

    return outFile, nil
}

func joinScripts(scripts []string) string {
    result := ""
    for i, script := range scripts {
        if i > 0 {
            result += ","
        }
        result += script
    }
    return result
}

```

Listing 9: Przykładowy szkic wysyłki raportu e-mail.

```

package mailer

import (
    "fmt"
    "net/smtp"
    "strings"
)

func SendReport(smtpHost string, smtpPort int, sender string,
    password string, recipient string, subject string,
    textBody string, htmlBody string) error {

    boundary := "REPORT-BOUNDARY-123456"
    auth := smtp.PlainAuth("", sender, password, smtpHost)

    headers := []string{
        fmt.Sprintf("From: %s", sender),
        fmt.Sprintf("To: %s", recipient),
        fmt.Sprintf("Subject: %s", subject),
    }
}

```

```

    "MIME-Version: 1.0",
    fmt.Sprintf("Content-Type: multipart/alternative; boundary
        =%s", boundary),
    "",
    fmt.Sprintf("--%s", boundary),
    "Content-Type: text/plain; charset=UTF-8",
    "",
    textBody,
    fmt.Sprintf("--%s", boundary),
    "Content-Type: text/html; charset=UTF-8",
    "",
    htmlBody,
    fmt.Sprintf("--%s--", boundary),
    "",
}

msg := strings.Join(headers, "\r\n")
addr := fmt.Sprintf("%s:%d", smtpHost, smtpPort)

return smtp.SendMail(addr, auth, sender, []string{recipient},
    []byte(msg))
}

```

6.4 Propozycja lekkiego obrazu kontenera

Listing 10: Minimalistyczny obraz kontenera dla prototypu.

```

FROM golang:1.26.1-alpine AS builder

WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download

COPY cmd/ /app/cmd/
COPY internal/ /app/internal/
RUN CGO_ENABLED=0 GOOS=linux go build -o /scanner ./cmd/scanner

FROM alpine:3.22

RUN apk add --no-cache nmap nmap-scripts bash ca-certificates
WORKDIR /app
COPY configs/ /app/configs/
COPY profiles/ /app/profiles/
COPY templates/ /app/templates/
COPY container/entrypoint.sh /app/entrypoint.sh

```

```
COPY --from=builder /scanner /app/scanner
RUN chmod +x /app/entrypoint.sh
ENTRYPOINT ["/app/entrypoint.sh"]
```

7 Scenariusz działania systemu

1. Operator przygotowuje router i jednorazowo aktywuje obsługę kontenerów.
2. Przez SSH wykonuje jedną komendę instalacyjną pobierającą skrypt bootstrap.
3. Skrypt importuje konfigurację, ustawia harmonogram i pobiera obraz kontenera.
4. Kontener uruchamia się automatycznie po starcie urządzenia.
5. O ustalonej porze agent wykonuje skan zgodny z przypisanym profilem; w wariancie podstawowym jest to monitoring diagnostyczny, a w wariancie rozszerzonym może to być profil pogłębiany lub pentestowy uruchamiany na żądanie.
6. Wyniki są przetwarzane do postaci raportu.
7. Raport trafia e-mailem do operatora lub administratora.
8. Administrator analizuje wykryte ryzyka i planuje dalsze działania naprawcze.

8 Ocena wykonalności rozwiązania

Projekt jest wykonalny technicznie, ponieważ wszystkie kluczowe elementy posiadają odpowiedniki w dostępnych technologiach:

- RouterOS oferuje mechanizmy `fetch`, `scheduler` i `e-mail`, które umożliwiają pobieranie zasobów, automatyzację oraz wysyłanie raportów (MikroTik 2026e, 2020, 2026d);
- RouterOS 7 wspiera uruchamianie kontenerów na architekturach `arm`, `arm64` i `x86`, przy czym producent wskazuje konieczność ostrożnego podejścia do bezpieczeństwa obrazów oraz wystawiania usług kontenerowych (MikroTik 2026b, 2026a);
- Nmap i NSE stanowią dojrzały, dobrze udokumentowany zestaw narzędzi do eksploracji sieci i automatyzacji testów bezpieczeństwa (Nmap Project 2026a, 2026b);
- literatura dotycząca bezpieczeństwa środowisk domowych i IoT potwierdza zasadność automatyzacji oceny ryzyka dla urządzeń końcowych (Fernandes, Jung i Prakash 2016; Mosenia i Jha 2017; Amro 2020; Akhilesh i inni 2022; Duan i inni 2021; Everson i Cheng 2024).

Największymi ryzykami wdrożeniowymi pozostają: ograniczone zasoby sprzętowe słabszych routerów, konieczność ostrożnego doboru profili skanowania, dostępność pamięci masowej na dane tymczasowe, potrzeba bezpiecznego przechowywania poświadczeń SMTP oraz

ryzyko destabilizacji wybranych usług przy użyciu profili obejmujących kategorie `intrusive`, `exploit` i `dos`.

9 Podsumowanie i zadeklarowane cele do osiągnięcia

W niniejszym dokumencie przedstawiono kompletny projekt aplikacji przeznaczonej do automatycznego nadzoru lokalnej sieci komputerowej z poziomu routera MikroTik. Zapropowane rozwiązanie łączy model operatorskiego zarządzania urządzeniami brzegowymi z lekkim, kontenerowym agentem bezpieczeństwa wykorzystującym Nmap oraz profile NSE dobierane zależnie od scenariusza operacyjnego. Przyjęta architektura jest modularna, możliwa do wdrożenia i zgodna z wymaganiem minimalnej ingerencji użytkownika końcowego.

Zadeklarowane cele do osiągnięcia w dalszej części projektu:

- zbudowanie prototypowego obrazu kontenera z agentem skanującym,
- przygotowanie skryptu `install.rsc` dla RouterOS,
- skonfigurowanie automatycznego harmonogramu uruchamiania skanów,
- przygotowanie zestawu profili Nmap/NSE o zróżnicowanym poziomie intensywności,
- implementacja parsera wyników i silnika oceny ryzyka,
- przygotowanie szablonu raportu HTML i wersji tekstowej,
- integracja z modułem wysyłki e-mail,
- opublikowanie rozwiązania w repozytorium GitHub w formie umożliwiającej instalację jedną komendą przez SSH,
- wykonanie testów na środowisku z routerem MikroTik i przykładową siecią lokalną,
- ocena wpływu rozwiązania na zasoby urządzenia brzegowego.

Bibliografia

- Akhilesh, Rohit, Oliver Bills, Naveen Chilamkurti i Mohammad Javed Morshed Chowdhury. 2022. „Automated Penetration Testing Framework for Smart-Home-Based IoT Devices”. *Future Internet* 14 (10): 276. <https://doi.org/10.3390/fi14100276>.
- Amro, Ahmed. 2020. „IoT Vulnerability Scanning: A State of the Art”. W *Computer Security*, redagował Sokratis K. Katsikas, Frédéric Cuppens, Nora Cuppens, Costas Lambrinoudakis, Christos Kalloniatis, John Mylopoulos, Alicia Antó, Stefanos Gritzalis, Weizhi Meng i Steven Furnell, 12501:84–99. Lecture Notes in Computer Science. Cham: Springer International Publishing. https://doi.org/10.1007/978-3-030-64330-0_6.
- Bouwmeester, Brennen, Elsa Rodríguez, Carlos Gañán, Michel van Eeten i Simon Parkin. 2021. „The Thing Doesn't Have a Name": Learning from Emergent Real-World Interventions in Smart Home Security”. W *Seventeenth Symposium on Usable Privacy and Security (SOUPS 2021)*, 493–512. USENIX Association. <https://www.usenix.org/conference/soups2021/presentation/bouwmeester>.
- Broadband Forum. Marzec 2018. *TR-069: CPE WAN Management Protocol*. Sprawozdanie techniczne. Broadband Forum. Termin wizyty 18 marca 2026. <https://www.broadband-forum.org/pdfs/tr-069-1-6-0.pdf>.
- . Czerwiec 2025. *TR-369: The User Services Platform. Issue 1 Amendment 4 Corrigendum 1*. Sprawozdanie techniczne. Broadband Forum. Termin wizyty 3 kwietnia 2026. <https://www.broadband-forum.org/pdfs/tr-369-1-4-1.pdf>.
- Duan, Xuanyu, Mengmeng Ge, Triet Huynh Minh Le, Faheem Ullah, Shuang Gao, Xuequan Lu i Muhammad Ali Babar. 2021. „Automated Security Assessment for the Internet of Things”. W *2021 IEEE 26th Pacific Rim International Symposium on Dependable Computing (PRDC)*, 47–56. IEEE. <https://doi.org/10.1109/PRDC53464.2021.00016>.
- Everson, Douglas i Long Cheng. 2024. „A Survey on Network Attack Surface Mapping”. *Digital Threats: Research and Practice* 5 (1): 1–25. <https://doi.org/10.1145/3640019>.
- Fernandes, Earlence, Jaeyeon Jung i Atul Prakash. 2016. „Security Analysis of Emerging Smart Home Applications”. W *2016 IEEE Symposium on Security and Privacy (SP)*, 636–654. IEEE. <https://doi.org/10.1109/SP.2016.44>.
- Kumar, Deepak, Kelly Shen, Benton Case, Deepali Garg, Galina Alperovich, Dmitry Kuznetsov, Rajarshi Gupta i Zakir Durumeric. 2019. „All Things Considered: An Analysis of IoT Devices on Home Networks”. W *28th USENIX Security Symposium (USENIX Security 19)*, 1169–1185. Santa Clara, CA: USENIX Association. <https://www.usenix.org/conference/usenixsecurity19/presentation/kumar-deepak>.
- MikroTik. 2020. „Scheduler - RouterOS”. Termin wizyty 22 marca 2026. <https://help.mikrotik.com/docs/spaces/ROS/pages/40992881/Scheduler>.
- . 2026a. „Container - HAProxy”. Termin wizyty 29 marca 2026. <https://help.mikrotik.com/docs/spaces/ROS/pages/297795651/Container%2B-%2BHAProxy>.

- MikroTik. 2026b. „Container - RouterOS”. Termin wizyty 17 marca 2026. <https://help.mikrotik.com/docs/spaces/ROS/pages/84901929/Container>.
- . 2026c. „Containerized App management - RouterOS”. Termin wizyty 21 marca 2026. <https://help.mikrotik.com/docs/spaces/ROS/pages/343244823/Containerized%2BApp%2Bmanagement>.
- . 2026d. „E-mail - RouterOS”. Termin wizyty 28 marca 2026. <https://help.mikrotik.com/docs/spaces/ROS/pages/24805377/E-mail>.
- . 2026e. „Fetch - RouterOS”. Termin wizyty 25 marca 2026. <https://help.mikrotik.com/docs/spaces/ROS/pages/8978514/Fetch>.
- . 2026f. „hAP ac²”. Termin wizyty 4 kwietnia 2026. https://mikrotik.com/product/hap_ac2.
- . 2026g. „hAP ax²”. Termin wizyty 4 kwietnia 2026. https://mikrotik.com/product/hap_ax2.
- . 2026h. „hEX refresh”. Termin wizyty 4 kwietnia 2026. https://mikrotik.com/product/hex_2024.
- Mosenia, Arsalan i Niraj K. Jha. 2017. „A Comprehensive Study of Security of Internet-of-Things”. *IEEE Transactions on Emerging Topics in Computing* 5 (4): 586–602. <https://doi.org/10.1109/TETC.2016.2606384>.
- Nmap Project. 2026a. „Nmap Reference Guide”. Termin wizyty 31 marca 2026. <https://nmap.org/book/man.html>.
- . 2026b. „Nmap Scripting Engine (NSE)”. Termin wizyty 1 kwietnia 2026. <https://nmap.org/book/man-nse.html>.
- . 2026c. „NSE Usage and Examples”. Termin wizyty 2 kwietnia 2026. <https://nmap.org/book/nse-usage.html>.
- Vardakis, George, George Hatzivasilis, Eleftheria Koutsaki i Nikos Papadakis. 2024. „Review of Smart-Home Security Using the Internet of Things”. *Electronics* 13 (16): 3343. <https://doi.org/10.3390/electronics13163343>.